

INSTITUT POLYTECHNIQUE DES SCIENCES AVANCÉES

Real-Time Embedded System : Final Assignement : Schedule search

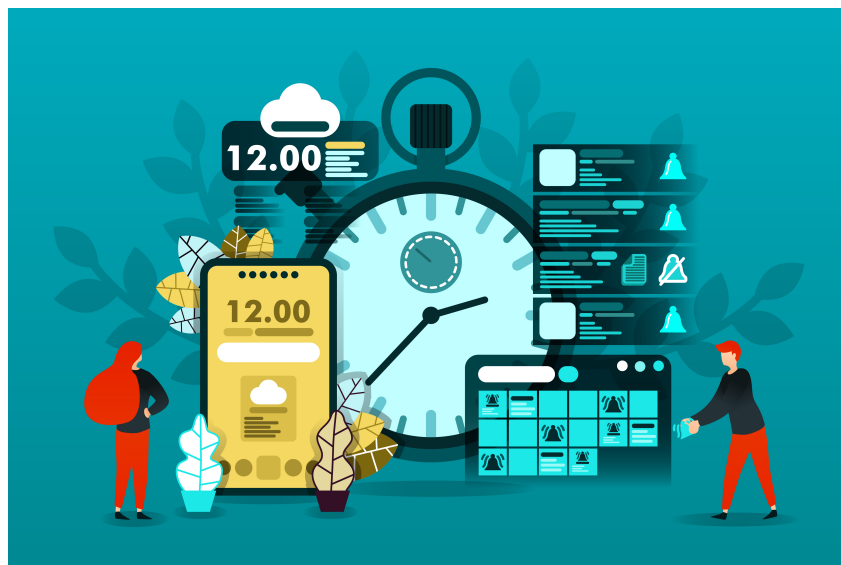


FIGURE 1 – Schedulability illustration

Class : 4TS2

Teacher : J. SIGNH

Due date : Wednesday 28th April 2026

Table des matières

1	Introduction	2
2	Schedulability analysis of our task set	3
2.1	Utilization Test	3
2.2	Non-Preemptive EDF Schedule	4
2.3	τ_5 allowed missing its deadline	6

1 Introduction

In real-time systems, the schedulability of the tasks is a crucial point. In terms of security, for example we do not want to have some critical tasks running at the same time and competing for the same resources, but more generally in terms of feasibility. This project focuses on the analysis and scheduling of a set of periodic tasks under timing constraints as shown below :

Task	C_i	T_i
τ_1	C_1	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

TABLE 1 – Task set description

Our first task τ_1 , with a period of 10, consists of computing the product of large randomly generated numbers. For the moment, we don't know its duration, we will estimate it experimentally. To do this, we will perform 50 measurements and compute statistical indicators such as the minimum, maximum and quartiles (Q1,Q2,Q3) and the Worst Case Execution Time (WCET) that we will call C_1 .

Once the execution time of τ_1 is determined, the complete task set is analyzed to evaluate its schedulability. A non-preemptive scheduling algorithm based on the Earliest Deadline First (EDF) policy is then implemented at the job level.

The objective is to create a schedule where no deadlines are missed while minimizing total waiting time of all jobs. Additionally, we had a schedule where τ_5 can miss its deadline in order to reduce waiting time.

2 Schedulability analysis of our task set

Now that we know the context of our project, we can run our code if our task set is schedulable or not. We had different constraints to respect and the objective will be to see if we respected all of them :

- No deadline missed
- Total waiting (delay because another job is executing) across all jobs is minimum.

We are also asked to use a non-preemptive algorithm to schedule this set of task. We choose EDF (Earliest Deadline First). The scheduling algorithm builds all jobs over the hyperperiod and applies a non-preemptive EDF policy. Let n be the number of tasks and J the total number of jobs over the hyperperiod. At each scheduling step, the algorithm scans the list of remaining jobs to find the earliest deadline, which has a complexity of $O(J)$. Since this operation is repeated for each job, the overall complexity is : $O(O(J^2))$. The different assumptions we made for this project were :

- The execution time C_1 is obtained experimentally using multiple measurements.
- The hyperperiod is used as the scheduling interval since it represents the smallest interval after which the task pattern repeats.
- Scheduling is non-preemptive.
- Implicit deadlines are given by the period of each task.
- EDF is used as the schedule policy.

Lets run our code to see the results.

The benchmark of task τ_1 was executed with 50 samples. The obtained execution-time statistics are presented in Table 2.

Metric	Execution time (ms)
Min	0.932079
Q1	0.953251
Q2	0.961639
Q3	0.977190
Max	1.369008
WCET	1.369008

TABLE 2 – Execution-time statistics for task τ_1

The worst observed execution time is therefore :

$$WCET = 1.369008$$

Since execution times are rounded up to the next integer value for the scheduling analysis, the execution time of task τ_1 is :

$$C_1 = \lceil WCET \rceil = \lceil 1.369008 \rceil = 2$$

The resulting task set is shown in Table 3.

2.1 Utilization Test

The processor utilization is computed as :

Task	C_i	T_i
τ_1	2	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

TABLE 3 – Task set after measuring C_1

$$U = \sum_i \frac{C_i}{T_i}$$

Thus :

$$U = \frac{2}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80}$$

$$U = 0.8375$$

Since :

$$U \leq 1$$

the processor has enough capacity to execute the task set.

The hyperperiod is :

$$H = \text{lcm}(10, 20, 40, 80) = 80$$

Therefore, the schedule is computed over the interval $[0, 80]$.

2.2 Non-Preemptive EDF Schedule

A non-preemptive Earliest Deadline First (EDF) policy was used. At each scheduling decision, the available job with the earliest deadline is selected. Once a job starts executing, it cannot be interrupted.

The obtained schedule is shown in Table 4.

Job	Start time	Finish time
τ_1^1	0	2
τ_2^1	2	5
τ_3^1	5	7
τ_4^1	7	9
τ_5^1	9	11
τ_1^2	11	13
τ_2^2	13	16
τ_6^1	16	18
τ_7^1	18	21
τ_1^3	21	23
τ_2^3	23	26
τ_3^2	26	28
τ_4^2	28	30
τ_1^4	30	32
τ_2^4	32	35
Idle	35	40
τ_1^5	40	42
τ_2^5	42	45
τ_3^3	45	47
τ_4^3	47	49
τ_5^2	49	51
τ_1^6	51	53
τ_2^6	53	56
τ_6^2	56	58
Idle	58	60
τ_1^7	60	62
τ_2^7	62	65
τ_3^4	65	67
τ_4^4	67	69
Idle	69	70
τ_1^8	70	72
τ_2^8	72	75
Idle	75	80

TABLE 4 – Non-preemptive EDF schedule over one hyperperiod

All jobs complete before their respective deadlines. Therefore, the task set is schedulable under the non-preemptive EDF scheduling policy.

The total processor busy time over the hyperperiod is :

$$C_{\text{busy}} = U \times H = 0.8375 \times 80 = 67$$

Thus, the total idle time is :

$$C_{\text{idle}} = H - C_{\text{busy}} = 80 - 67 = 13$$

This confirms that the task set is schedulable and that the processor still has available idle time within the hyperperiod.

2.3 τ_5 allowed missing its deadline

In the previous section, we saw that everything was schedulable with a non-preemptive EDF policy, ensuring that every deadlines were not missed. However it could be underoptimal in terms of waiting time. In many applications, the different tasks don't have the same importance, then some of them could pass their deadlines and it won't be a problem. Now, we will consider τ_5 less critical and we will tolerate if this task miss the deadline if necessary. The objective is to reduce the total waiting time prioritizing more important task. The code is exactly the same but we apply a penalty for the τ_5

Job	Start	Finish
τ_1^1	0	2
τ_2^1	2	5
τ_3^1	5	7
τ_4^1	7	9
τ_6^1	9	11
τ_1^2	11	13
τ_2^2	13	16
τ_5^1	16	18
τ_7^1	18	21
τ_1^3	21	23
τ_2^3	23	26
τ_3^2	26	28
τ_4^2	28	30
τ_1^4	30	32
τ_2^4	32	35
Idle	35	40
τ_1^5	40	42
τ_2^5	42	45
τ_3^3	45	47
τ_4^3	47	49
τ_6^2	49	51
τ_1^6	51	53
τ_2^6	53	56
τ_5^2	56	58
Idle	58	60
τ_1^7	60	62
τ_2^7	62	65
τ_3^4	65	67
τ_4^4	67	69
Idle	69	70
τ_1^8	70	72
τ_2^8	72	75

TABLE 5 – Schedule obtained with relaxed τ_5 priority

The standard EDF schedule and the relaxed τ_5 schedule both produce feasible schedules over the hyperperiod $H = 80$. In both cases, all jobs meet their deadlines.

The main difference is the execution order of some jobs. In the relaxed τ_5 schedule, task τ_5 is assigned a lower priority, which delays its execution compared to the standard EDF schedule.

For example, in the standard EDF schedule :

$$\tau_5^1 : [9, 11)$$

whereas in the relaxed schedule :

$$\tau_5^1 : [16, 18)$$

Similarly, τ_5^2 is executed later in the relaxed schedule :

$$\tau_5^2 : [56, 58)$$

Despite this delay, both jobs of τ_5 still meet their deadlines :

$$R_{\tau_5^1} = 18 \leq 40$$

$$R_{\tau_5^2} = 18 \leq 40$$

Therefore, allowing τ_5 to miss its deadline was not necessary in this case, since the relaxed schedule still satisfies all deadlines.

Concerning the response time and waiting time, despite the rule about τ_5 , the waiting time is still the same : $W = 140$. I tried with different rules and penalties on τ_5 , but the waiting time is always the same. This shows that allowing τ_5 to miss its deadline does not improve the performance for this task set.

Job	R_i	D_i	Finish	Status
τ_1^1	2	10	2	OK
τ_2^1	5	10	5	OK
τ_3^1	7	20	7	OK
τ_4^1	9	20	9	OK
τ_6^1	11	40	11	OK
τ_1^2	3	10	13	OK
τ_2^2	6	10	16	OK
τ_5^1	18	40	18	OK
τ_7^1	21	80	21	OK
τ_3^3	3	10	23	OK
τ_3^2	6	10	26	OK
τ_3^3	8	20	28	OK
τ_4^2	10	20	30	OK
τ_4^3	2	10	32	OK
τ_2^4	5	10	35	OK
τ_5^2	2	10	42	OK
τ_1^3	5	10	45	OK
τ_3^3	7	20	47	OK
τ_3^4	9	20	49	OK
τ_6^2	11	40	51	OK
τ_6^3	3	10	53	OK
τ_6^4	6	10	56	OK
τ_2^2	18	40	58	OK
τ_7^2	2	10	62	OK
τ_2^4	5	10	65	OK
τ_3^4	7	20	67	OK
τ_4^4	9	20	69	OK
τ_8^1	2	10	72	OK
τ_2^5	5	10	75	OK

TABLE 6 – Response-time verification with relaxed τ_5